

Design a Rate Limiter - Distributed Environment

Building a rate limiter on a single server is straightforward.
However, scaling to **multiple servers and concurrent threads** introduces challenges.

1. Key Challenges

- Race Condition
- Synchronization Issue

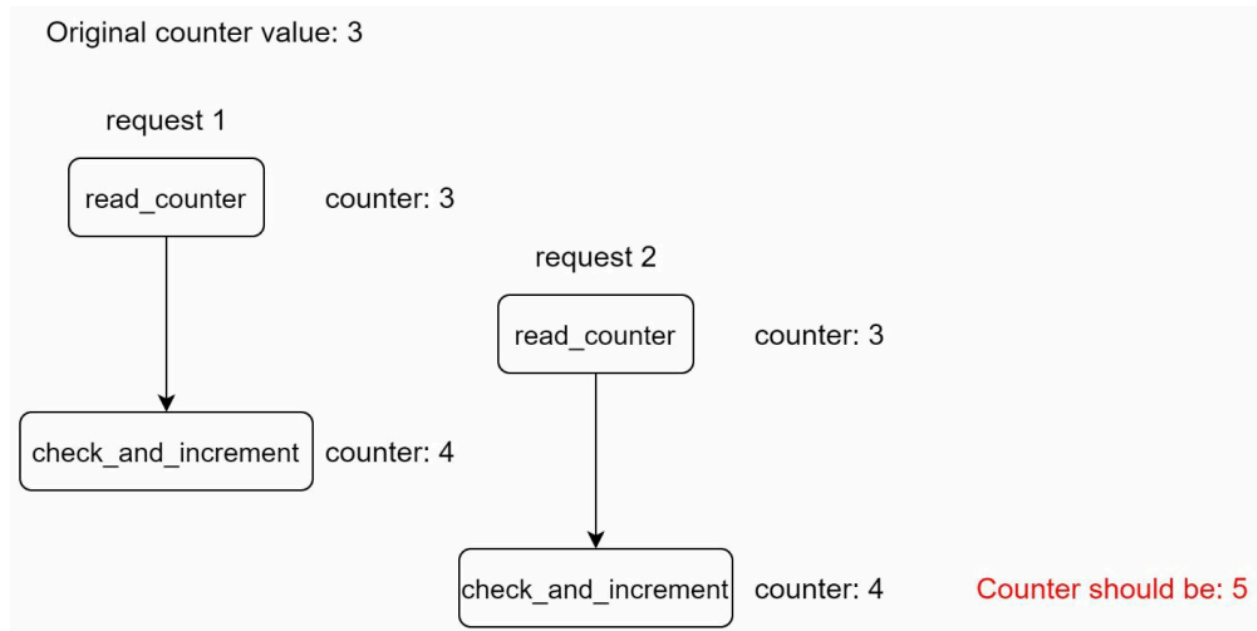
2. Race Condition

How Rate Limiter Works (Recap)

None

1. Read counter from Redis
2. Check if (counter + 1) exceeds limit
3. If not → increment counter

Problem Scenario



- Assume counter = 3
- Two requests arrive **concurrently**

Flow:

- Both read counter = 3
- Both increment → write back 4

Final value becomes **4 (incorrect)**

Correct value should be **5**

Naive Solution

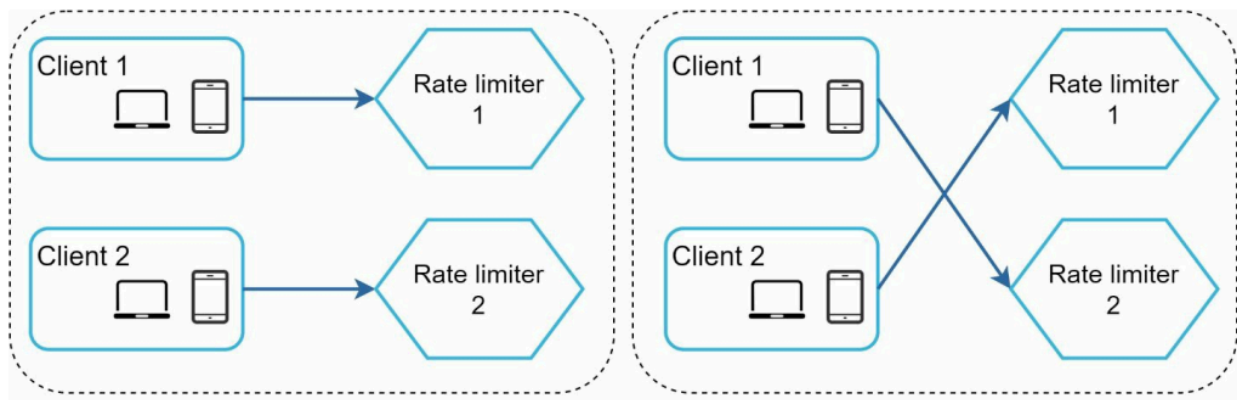
- Use **locks**
- Problem:
 - Slows down system significantly

Better Solutions

- Lua scripts
- Sorted Sets in Redis

3. Synchronization Issue

Problem Scenario



- Multiple rate limiter servers:
 - Client 1 → Rate Limiter 1
 - Client 2 → Rate Limiter 2
- Due to stateless web tier:
 - Requests may go to different servers

Result:

- Rate limiter does not have **complete request history**
- Leads to incorrect rate limiting

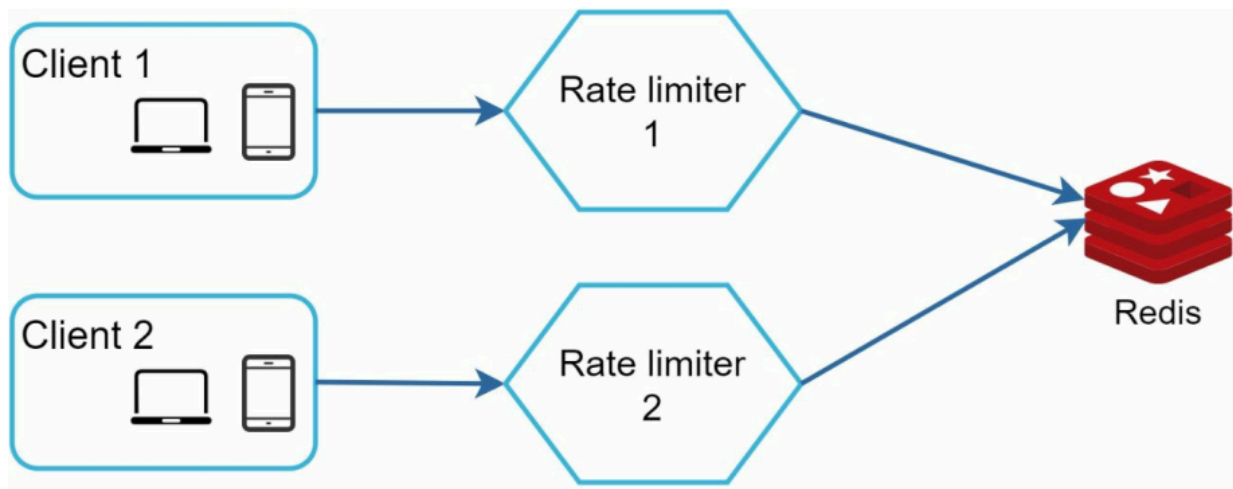
Naive Solution: Sticky Sessions

- Route same client to same server

Problems:

- Not scalable
- Not flexible

Better Solution: Centralized Data Store



- Use centralized storage (e.g., Redis)
- All rate limiter instances share same data

4. Key Takeaway

In distributed systems:

- Avoid race conditions using **atomic operations**
- Ensure synchronization via **centralized data store**
- Avoid solutions that reduce scalability (like sticky sessions)